

CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 07/Unit 07: Linked Data Structures

Faisal Alvi

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk)



Unit Outline

1. Linked Data Structures
2. The class Node
3. The Linked List Class
4. Functions to Print a Linked List
5. Functions to Add Elements
6. Functions to Remove Elements
7. Other Linked Data Structures



Linked List – Motivation

- Consider using an array for storing integers.
- In an array of n elements, what is the time complexity of inserting an element at the 2nd position (i.e. at index 1, if the indices start from 0).
- What is the time complexity if you have to insert an element at the 2nd last position, (i.e. at index $n - 2$, if the last index is $n - 1$?).
- Finally, in C++ do you have to declare the size of an array, even if it is dynamically allocated?



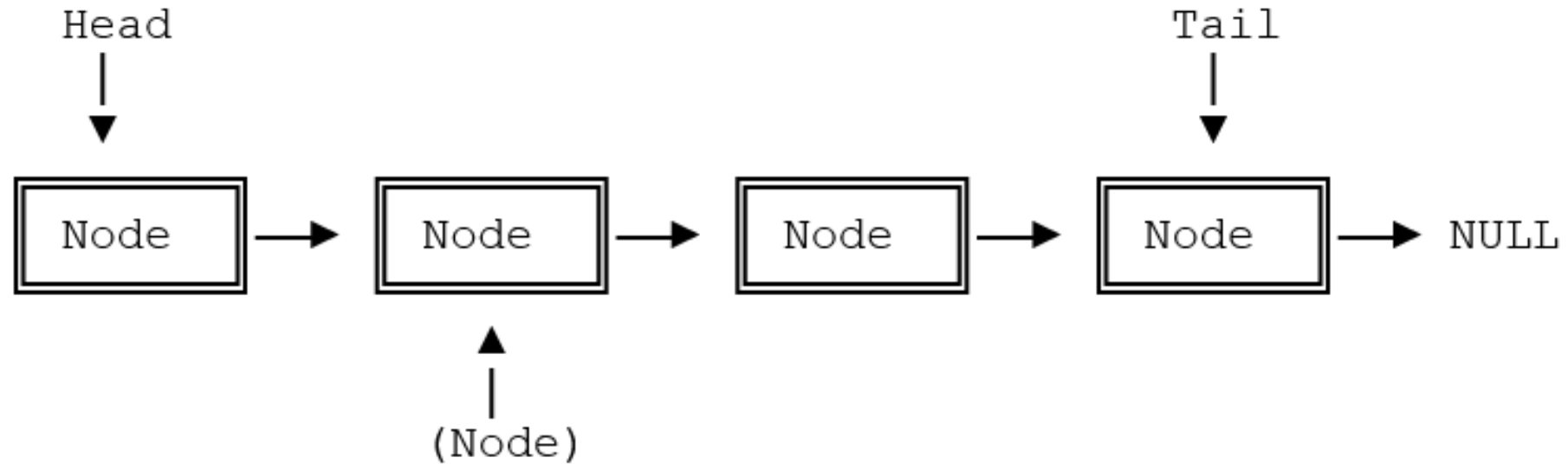


Linked List

- A linked list is a linked data structure – a collection of components, called nodes.
- Every node (except the last node) contains the address of the next node.
- Thus, every node in a linked list has two components:
 - **(data)** to store the relevant information,
 - **(node)** to store the address, called the link, of the next node in the list.



Linked List





Linked List – the class Node

- At the foundation of every Linked Data Structure lies the class Node.
- The Node defines what a node was shown in the previous slide – data and a link.
- Here's the code →

```
class Node {  
public:  
    int data;    //data  
    Node* next;    //link  
  
    Node() : data(0), next(nullptr)  
    {  
    }  
  
    Node(int val) {  
        data = val;  
        next = nullptr;  
    }  
};
```



The -> notation

- In C++, the -> symbol is called the arrow operator.
- It is primarily used to access members (attributes or methods) of an object or a struct through a pointer.
- Use: When you have a pointer to an object or a struct, instead of dereferencing the pointer and then accessing the member with the dot (.) operator, you can use the -> operator as a shorthand.
- Example: Let's say you have a struct called Point with two members, x and y.

```
struct Point
{
    int x;
    int y;
};
```



The -> notation

- If you have a pointer to a Point object, you can access its members as follows:

```
Point* p = new Point;
```

```
p->x = 10; // Using the arrow operator to access  
         'x'
```

```
p->y = 20; // Using the arrow operator to access  
         'y'
```

```
cout << "x: " << p->x << ", y: " << p->y << endl;
```

- Without the arrow operator, this will be:

```
(*p).x = 10;
```

```
(*p).y = 20;
```



Linked List – the class (and the -> notation)

```
class LinkedList {  
private:  
    Node* head; // Head of the list  
    Node* tail; // Tail of the list  
    //Note that we are using a pointer to a node object  
public:  
    // Constructor to initialize an empty linked list  
    LinkedList() : head(nullptr), tail(nullptr) {  
    }  
}
```



```

void printList() {
    Node* current = head; // Start from the head of the list
    if (current == nullptr) {
        cout << "List is empty." << endl;
        return;
    }
    while (current != nullptr) {
        cout << current->data << " -> ";
        current = current->next;
    }
    cout << "NULL" << endl;
}

```

Linked List
Class – print
the elements

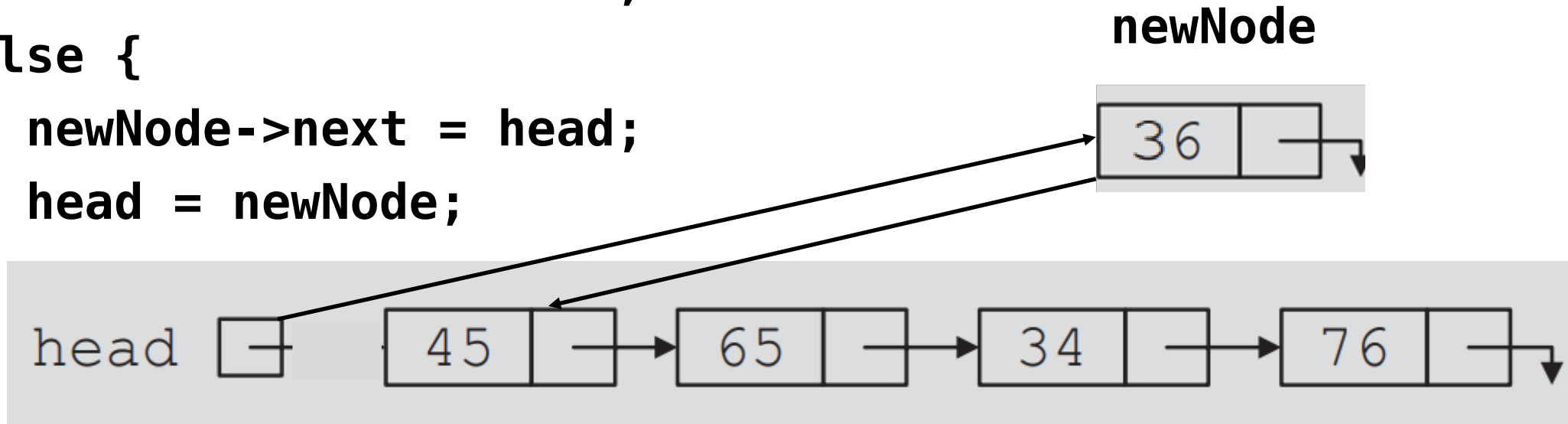
45 -> 35 -> 64 -> 76 -> NULL

//Can this function be modified to return the number of elements?



Linked List – Add Method – Add to Front

```
void addElement(int value) {  
    Node* newNode = new Node(value);  
    if (head == nullptr) {  
        head = tail = newNode;  
    } else {  
        newNode->next = head;  
        head = newNode;  
    }  
}
```





Linked List – Add to Rear Function

Class Exercise

- Can you write `addtoRear(int value)` function?
- Make sure to take care of all special cases. In particular you will have to:
- Think about the case when the list is empty.
- If non-empty, will you need to adjust the tail?
- If non-empty, will you need to adjust the head?
- Any other special cases?



Linked List – Remove from Front

```
void removeElement() {  
    if (head == nullptr) {  
        cout << "List is empty, nothing to remove." << endl;  
        return;  
    }  
    Node* temp = head; // Store the current head  
    head = head->next; // Move head to the next node  
    delete temp; // Free memory of the old head  
    if (head == nullptr) {  
        //If the list becomes empty, update tail to nullptr  
        as well  
        tail = nullptr;  
    }  
}
```



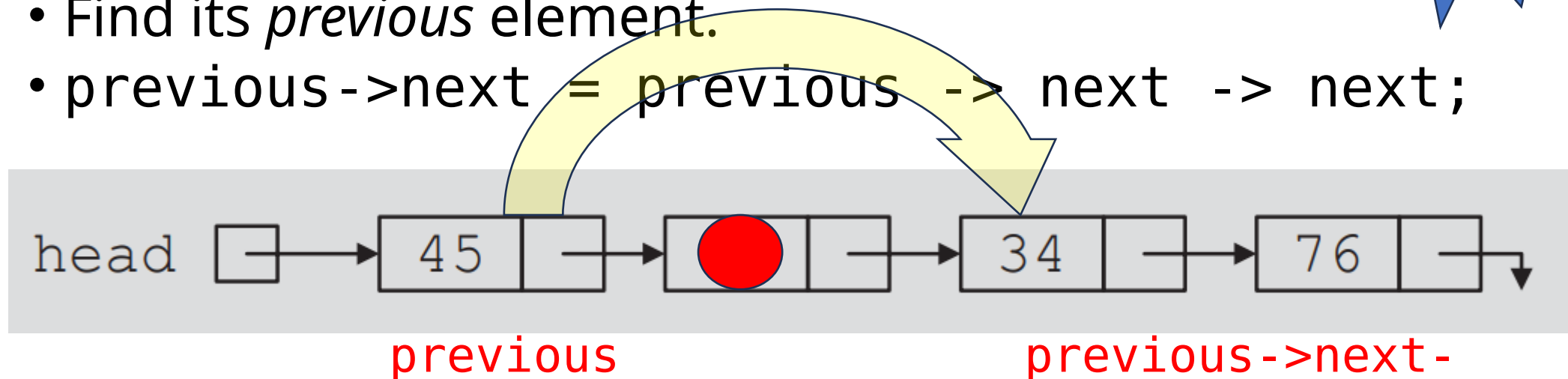
Linked List – Remove from rear

Class Exercise

- Can you write `removeFromRear()` function?
- Make sure to take care of all special cases. In particular you will have to:
- Think about the case when the list is empty.
- If non-empty, will you need to adjust the tail?
- If non-empty, will you need to adjust the head?
- Any other special cases?

LinkedList – General Delete Function

- How will you delete an *arbitrary* element from a linked list
- Assume the element already exists in the list!
- Hint:
 - Find the element to be deleted
 - Find its *previous* element.
 - `previous->next = previous->next->next;`





Linked List – the find(int value) function

- Observe that to delete a given element from the linked list (from the previous slide), we had to first find it!
- The declaration for this function could be:
- `Node* find(int value);`
- Can you write the code for this?



Other Linked Data Structures

- Apart from Linked Lists, there are other linked data structures that can be:
 - created from the class Node,
 - or created using the LinkedList class.
- Examples are:
 - Queue,
 - Stack,
 - Double ended queue (or Deque)



Summary

- In this session, we had an introduction to linked lists.
- Linked list is a linked data structure that is based on *nodes* – data element with *data* and link.
- Linked lists typically support add, delete, find and remove functions at various ends of the list.
- Other linked data structures can also be made such as stacks, queues and double ended queues.